

Lập Trình Hàm

Guo Jiabao
byvoid@byvoid.com

Nguồn gốc: Lập trình hàm

Others / Functional Programming - Guo Jiabao

Abstract

Bài trình bày lập trình hàm từ hai góc nhìn. Phần đầu dùng bài toán dừng, phép tính lambda, điểm bất động và tổ hợp tử Y để giải thích vì sao đệ quy có thể được biểu diễn bằng một hệ hình thức rất nhỏ. Phần sau chuyển sang thực hành với Haskell: danh sách, khớp mẫu, đệ quy, tính lười, hàm bậc cao, rồi kết thúc bằng các đặc trưng lập trình hàm đã đi vào các ngôn ngữ phổ biến như Python, JavaScript, C# và C++11.

Từ khóa. Lập trình hàm; phép tính lambda; bài toán dừng; tổ hợp tử Y; Haskell; tính lười; hàm bậc cao; bao đóng.

Contents

1	Lập trình hàm là gì?	2
2	Từ bài toán dừng	2
3	Phép tính lambda	3
3.1	Hai tiên đề cơ bản	3
3.2	Biểu thức điều kiện	4
4	Đệ quy và tổ hợp tử Y	4
4.1	Tham số hóa chính hàm đang định nghĩa	5
4.2	Điểm bất động	5
4.3	Tổ hợp tử Y	6
4.4	Tương đương Turing	6
5	Lập trình hàm trong thế giới thực	6
5.1	Haskell	6
5.2	Chương trình Haskell đầu tiên	7
5.3	Danh sách	7
5.4	Khớp mẫu	7
5.5	Tính tổng danh sách	7

5.6	Kiểm tra palindrome	8
5.7	Xóa các phần tử lặp liên tiếp	8
6	Tính lười	8
6.1	Dãy Fibonacci	9
7	Một số ví dụ Haskell khác	9
7.1	Sắp xếp nhanh	9
7.2	Biểu diễn cây nhị phân	10
7.3	Duyệt trung thứ tự	10
7.4	Chiều cao cây	10
7.5	Hàm bậc cao	10
8	Ý tưởng hàm trong kỹ nghệ phần mềm	11
8.1	Hàm ẩn danh	11
8.2	Bao đóng	11
8.3	Dùng bao đóng để currying	12
9	Kết luận	12

1 Lập trình hàm là gì?

Hầu như ai học lập trình cũng từng nghe về “hướng đối tượng”: đóng gói, kế thừa, đa hình, và nói chung là nghe có vẻ rất mạnh. Ngược lại, cái tên “lập trình hàm” dễ tạo cảm giác đơn giản quá mức.

Thực ra lập trình hàm là một hệ hình lập trình hoàn toàn khác. Đối tượng đối lập tự nhiên của nó không phải hướng đối tượng, mà là **lập trình mệnh lệnh**. Một chương trình mệnh lệnh mô tả một dãy thao tác làm thay đổi trạng thái; một chương trình hàm cố gắng mô tả quan hệ tính toán bằng biểu thức và hàm. Những đặc trưng thường gặp là:

- giá trị bất biến;
- tính lười;
- hàm bậc cao;
- không có tác dụng phụ quan sát được;
- mọi thứ đều có thể được nhìn như hàm.

2 Từ bài toán dừng

Khi gõ lỗi, ta thường gặp vòng lặp vô hạn. Một ý tưởng rất hấp dẫn là tạo ra một công cụ tự động kiểm tra xem một chương trình có rơi vào vòng lặp vô hạn hay không. Đáng tiếc, công cụ tổng quát như vậy không tồn tại.

Bài toán dừng hỏi: cho một chương trình bất kỳ và đầu vào của nó, hãy quyết định chương trình đó có kết thúc sau hữu hạn bước tính hay không.

Giả sử thật sự có một thuật toán quyết định dừng. Ta mô tả nó bằng mã giả:

```
function halting(func, input) {
    return if_func_will_halt_on_input;
}
```

Nghĩa là khi đưa vào một hàm và dữ liệu đầu vào của hàm đó, thuật toán sẽ trả lời hàm có kết thúc hay không.

Bây giờ dựng một hàm khác:

```
function paradox(func) {
    if (halting(func, func)) {
        for (;;) {
            // loop forever
        }
    }
}
```

Sau đó gọi:

```
paradox(paradox)
```

Nếu `halting(paradox, paradox)` nói rằng lời gọi này sẽ dừng, thân hàm lại đi vào vòng lặp vô hạn. Nếu nó nói rằng lời gọi này không dừng, điều kiện `if` không chạy và hàm kết thúc ngay. Cả hai khả năng đều mâu thuẫn. Vì vậy bài toán dừng là không quyết định được.

Ví dụ này chỉ là phần dẫn nhập. Phần chính của bài là phép tính lambda, một hệ hình thức rất nhỏ nhưng đủ để mô tả tính toán.

3 Phép tính lambda

Cú pháp của phép tính lambda chỉ có ba dạng:

$$\begin{aligned} \langle \text{biểu thức} \rangle &::= \langle \text{định danh} \rangle, \\ \langle \text{biểu thức} \rangle &::= \lambda \langle \text{một hoặc nhiều định danh} \rangle. \langle \text{biểu thức} \rangle, \\ \langle \text{biểu thức} \rangle &::= (\langle \text{biểu thức} \rangle \langle \text{biểu thức} \rangle). \end{aligned}$$

Ví dụ:

$$\lambda x y. x + y.$$

Hai dạng đầu dùng để sinh ra hàm, còn dạng thứ ba dùng để gọi hàm. Chẳng hạn:

$$((\lambda x y. x + y) 2 3).$$

Để viết gọn, ta có thể đặt:

$$\text{let } \text{add} = \lambda x y. x + y,$$

rồi gọi $(\text{add } 2 \ 3)$.

3.1 Hai tiên đề cơ bản

Tiên đề đổi tên cho phép thay tên biến bị ràng buộc mà không đổi ý nghĩa:

$$\lambda x y. x + y \Rightarrow \lambda a b. a + b.$$

Tiên đề thay thế cho phép áp dụng hàm vào đối số:

$$(\lambda x y. x + y) a b \Rightarrow a + b.$$

Hai quy tắc đơn giản này đã tạo thành lõi của phép tính lambda.

Có thể xem phép tính lambda như một máy sinh hàm. Ví dụ:

```
let mul =  $\lambda x y. x * y$ ,  
let con =  $\lambda x y. xy$ .
```

Khi thay đối số vào, ta nhận được:

$$\text{mul } 3 \ 5 \Rightarrow 3 * 5, \quad \text{con 'BYV' 'oid'} \Rightarrow \text{'BYVoid'}.$$

3.2 Biểu thức điều kiện

Xét hàm phủ định:

```
not false  $\rightarrow$  true,  
not true   $\rightarrow$  false.
```

Phép and có thể được định nghĩa theo bảng chân trị:

```
and true true   $\rightarrow$  true,  
and true false  $\rightarrow$  false,  
and false true   $\rightarrow$  false,  
and false false  $\rightarrow$  false.
```

Viết tổng quát hơn:

```
and true value  $\rightarrow$  value,  
and false value  $\rightarrow$  false,  
and value true   $\rightarrow$  value,  
and value false  $\rightarrow$  false.
```

Từ đó có thể định nghĩa if:

```
if =  $\lambda$  cond tvalue fvalue. (cond and tvalue)  
                        or (not cond and fvalue).
```

Khi cond = true, ta có:

```
if true a b  $\rightarrow$  (true and a) or (not true and b)  
               $\rightarrow$  a or false  
               $\rightarrow$  a.
```

4 Độ quy và tổ hợp tử Y

Ta muốn tính $n!$ bằng định nghĩa quen thuộc:

```
let fact =  $\lambda n. \text{if } (n == 0) \ 1 \ (n * \text{fact } (n - 1))$ .
```

Vấn đề là trong lúc định nghĩa fact, ta đã tham chiếu đến chính fact. Các trình biên dịch thực tế thường hỗ trợ cách viết này, nhưng nó không trực tiếp nằm trong hệ tiên đề toán học chặt chẽ ở trên.

4.1 Tham số hóa chính hàm đang định nghĩa

Nếu không được tham chiếu đến chính mình, ta đưa “chính mình” thành một tham số:

```
let P = λ self n. if (n == 0) 1 (n * self(self (n - 1))).
```

Sau đó đặt:

```
let fact n = P (P n).
```

Khi tính 4!, quá trình có dạng:

```
fact 4 → P (P 4)
      → 4 * P (P 3)
      → 4 * 3 * P (P 2)
      → 4 * 3 * 2 * P (P 1)
      → 4 * 3 * 2 * 1.
```

Cách này chạy được, nhưng nó chưa phải đệ quy thật sự; mỗi lần gọi chỉ truyền thêm một tham số phụ.

4.2 Điểm bất động

Giả sử hàm đệ quy fact thật sự tồn tại. Quay lại dạng giả đệ quy:

```
let P = λ self n. if (n == 0) 1 (n * self (n - 1)).
```

Hàm P nhận hai tham số, nhưng ta có thể áp dụng một phần:

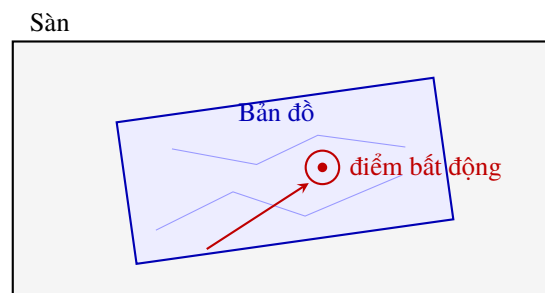
```
P(fact) → λ n. if (n == 0) 1 (n * fact (n - 1)).
```

Biểu thức về phải chính là fact, do đó:

```
P(fact) = fact.
```

Nói cách khác, fact là một **điểm bất động** của P: một giá trị mà khi đưa qua hàm vẫn thu được chính giá trị đó.

Trên slide gốc, tác giả minh họa bằng hình ảnh một tấm bản đồ rơi xuống sàn: trên bản đồ có một điểm trùng với vị trí thật của chính điểm đó ngoài đời. Sơ đồ dưới đây ghi lại ý tưởng đó: ánh xạ từ bản đồ ra sàn luôn có một điểm trên bản đồ khớp với đúng vị trí thật của nó.



Hình 1: Minh họa trực giác về điểm bất động: một điểm trên bản đồ rơi xuống sàn nằm đúng trên vị trí mà nó biểu diễn.

Nếu tìm được điểm bất động của hàm giả đệ quy P, ta sẽ biến nó thành hàm đệ quy thật sự.

4.3 Tổ hợp tử Y

Giả sử có một hàm kỳ diệu Y tìm được điểm bất động:

$$Y(F) = f = F(Y(F)).$$

Nếu $F(f) = f$, thì $Y(F)$ là một điểm bất động của F . Với hàm P ở trên, ta sẽ có:

$$Y(P) = \text{fact}.$$

Tổ hợp tử Y có thể được dựng như sau:

$$\begin{aligned} \text{let } Y &= \lambda F. G(G), \\ \text{trong đó } G &= \lambda \text{self}. F(\text{self}(\text{self})). \end{aligned}$$

Kiểm tra với P:

$$\begin{aligned} Y(P) &= G(G), \quad G = \lambda \text{self}. P(\text{self}(\text{self})) \\ &= P(G(G)) \\ &= \lambda n. \text{if } (n == 0) \ 1 \ (n * G(G) \ (n - 1)). \end{aligned}$$

Nếu đặt $Y(P) = \text{fact}$, ta nhận được:

$$\text{fact} = \lambda n. \text{if } (n == 0) \ 1 \ (n * \text{fact} \ (n - 1)),$$

đúng là hàm đệ quy mong muốn.

Vậy khi muốn định nghĩa một hàm đệ quy, ta chỉ cần thêm một tham số self, viết hàm theo kiểu giả đệ quy, rồi áp dụng tổ hợp tử Y. Tổ hợp tử này cũng là nguồn gốc tên của công ty Y Combinator ở Thung lũng Silicon. Nhà sáng lập Paul Graham còn nổi tiếng với cuốn *Hackers and Painters*.

4.4 Tương đương Turing

Việc suy ra tổ hợp tử Y trong hệ tiên đề của phép tính lambda tương đương với việc suy ra định lý: trong quá trình định nghĩa hàm, ta có thể tham chiếu đến chính hàm đó. Đây là một bước quan trọng để chứng minh phép tính lambda là **tương đương Turing**.

Điều đó có nghĩa là năng lực tính toán của phép tính lambda khớp với năng lực tính toán của máy tính. Bất kỳ chương trình nào cũng có thể được mô tả bằng phép tính lambda, và mọi hàm mô tả được bằng phép tính lambda đều có thể được máy tính tính toán.

Bài toán dừng cũng có một phát biểu tương đương trong phép tính lambda: không tồn tại thuật toán quyết định hai hàm lambda tùy ý có tương đương hay không, tức là có cho cùng kết quả với mọi đầu vào n hay không.

5 Lập trình hàm trong thế giới thực

Những phần trên thuộc về lý thuyết phép tính lambda, rất gần với cách suy nghĩ của nhà toán học. Bây giờ ta chuyển sang góc nhìn kỹ sư và xem lập trình hàm xuất hiện trong thực tế như thế nào.

5.1 Haskell

Haskell là một ngôn ngữ lập trình hàm thuần túy, được đặt tên để tưởng nhớ Haskell Curry. Tổ hợp tử Y vừa nói ở trên gắn với tên ông; ngoài ra ông còn đưa ra khái niệm **currying**, tức áp dụng một phần.

Trong Haskell, mọi thứ đều là hàm. Thậm chí Haskell không có khái niệm biến theo kiểu lập trình mệnh lệnh: một biến chỉ được gán một lần rồi không thay đổi, giống như cách biến được dùng trong suy luận toán học.

Haskell cũng không có cấu trúc điều khiển lặp thông thường như `for`; thay vào đó, ta dùng đệ quy. Hai đặc trưng quan trọng khác là không có tác dụng phụ và tính lười. Không có tác dụng phụ nghĩa là cùng một hàm với cùng đầu vào luôn cho cùng kết quả; tính lười nghĩa là biểu thức không được tính ngay nếu kết quả của nó chưa cần dùng.

5.2 Chương trình Haskell đầu tiên

Trước khi chạy Haskell, cần cài trình biên dịch GHC. Chạy `ghci` để vào chế độ tương tác. Ví dụ định nghĩa hàm lớn hơn:

```
let max a b = if a > b then a else b
```

```
Prelude> max 3 4
4
```

```
Prelude> max 1.0001 1
1.0001
```

```
Prelude> max "BYVoid" "CmYkRgB123"
"CmYkRgB123"
```

5.3 Danh sách

Danh sách trong Haskell có thể được mô tả bằng định nghĩa đệ quy:

$$\text{list } X ::= [] \mid \text{elem} : (\text{list } X).$$

Tức là:

$$\begin{aligned} \text{danh sách rỗng} &= [], \\ [1] &= 1: [], \\ [1, 2, 3] &= 1: 2: 3: []. \end{aligned}$$

Nhập `2:1:3:7:8:[]` trong Haskell sẽ thu được:

```
[2,1,3,7,8]
```

5.4 Khớp mẫu

Định nghĩa:

```
let first (elem:rest) = elem
```

Khi nhập `first [1,3]`, kết quả là 1. Ở đây `elem:rest` là mẫu của đối số hàm. Danh sách `[1,3]` thật ra là `1:3:[]`; `elem` khớp với 1, còn `rest` khớp với `3:[]`, tức `[3]`.

5.5 Tính tổng danh sách

Lưu nội dung sau vào `acc.hs`:

```
accumulate [] = 0
accumulate (elem:rest) = elem + accumulate rest
main = print (accumulate [1,2,3])
```

Chạy runghc acc.hs sẽ in ra:

6

5.6 Kiểm tra palindrome

```
palindrome [] = True
palindrome [_] = True
palindrome (elem:rest) =
    (elem == last rest) && (palindrome (init rest))
```

```
> palindrome [1, 2, 3, 2, 1]
True
> palindrome [1, 1, 2]
False
> palindrome "madam"
True
```

5.7 Xóa các phần tử lặp liên tiếp

```
cut cond [] = []
cut cond (elem:rest) =
    if cond elem then cut cond rest else elem:rest
```

```
compress [] = []
compress (elem:rest) = elem : compress (cut (== elem) rest)
```

```
> compress [1, 2, 2, 2, 3, 3]
[1, 2, 3]
> compress "aaabbaccc"
"abac"
```

6 Tính lười

Trong Haskell có thể định nghĩa danh sách vô hạn:

`[1..]` là mọi số nguyên dương,

`[1,3..]` là mọi số lẻ dương.

Trong đa số ngôn ngữ, điều này rất khó tưởng tượng vì chúng dùng tính toán sớm. Haskell dùng tính lười, nên danh sách chỉ được tính đến phần thật sự cần dùng. Ví dụ:

```
[1,3..] !! 42
```

trả về 85.

6.1 Dãy Fibonacci

Cách mô tả Fibonacci gần với toán học nhất là:

```
fib 0 = 1
fib 1 = 1
fib a = fib (a - 1) + fib (a - 2)
```

Không may là thuật toán này có độ phức tạp $O(2^N)$, và trình biên dịch Haskell không tự động biến nó thành đệ quy có ghi nhớ.

Ta có thể dùng danh sách vô hạn để viết thuật toán tuyến tính chỉ bằng một dòng:

```
fib = 1:1:zipWith (+) fib (tail fib)
```

Khi đó `fib !! 4` là 5, còn `fib !! 42` là 433494437. Giá trị `fib !! 1000` là:

```
70330367711422815821835254877183549770181269836358732742604905087154537118196933
57974224949456261173348775044924176599108818636326545022364710601205337412127386
7339111198139373125598767690091902245245323403501
```

Hàm `tail` trả về phần còn lại của danh sách sau khi bỏ phần tử đầu. Ví dụ `tail [1..]` trả về `[2..]`. Hàm `zipWith` nhận một hàm và hai danh sách, áp dụng hàm đó lên từng cặp phần tử tương ứng rồi trả về danh sách kết quả. Ví dụ:

```
zipWith (*) [2,3,5] [1,2,3]
```

trả về `[2,6,15]`.

Vì vậy:

```
fib = 1:1:zipWith (+) fib (tail fib)
```

sinh một danh sách vô hạn. Hai phần tử đầu là 1; các phần tử sau được tạo bằng cách cộng danh sách hiện tại với chính nó sau khi bỏ phần tử đầu:

```
[1, 1, 2, 3, 5, 8, 13, 21, ...] -- fib
+ [1, 2, 3, 5, 8, 13, 21, 34, ...] -- tail fib
= [2, 3, 5, 8, 13, 21, 34, 55, ...]
```

Nhờ tính lười, danh sách không được tính toàn bộ ngay lập tức; phần tử nào cần dùng mới được sinh ra.

7 Một số ví dụ Haskell khác

7.1 Sắp xếp nhanh

Trên slide gốc chỉ nêu nhánh đệ quy; để chương trình hoàn chỉnh, ta thêm trường hợp danh sách rỗng:

```
qsort [] = []
qsort (elem:rest) =
  (qsort lesser) ++ [elem] ++ (qsort greater)
  where
    lesser = filter (< elem) rest
    greater = filter (>= elem) rest
```

Toán tử `++` dùng để nối danh sách. Hàm `filter` trả về danh sách gồm những phần tử thỏa điều kiện.

7.2 Biểu diễn cây nhị phân

```
data Tree a = Empty | Node a (Tree a) (Tree a)
```

```
tree = Node 'd'
      (Node 'b'
        (Node 'a' Empty Empty)
        (Node 'c' Empty Empty)
      )
      (Node 'e'
        Empty
        (Node 'g'
          (Node 'f' Empty Empty)
          Empty
        )
      )
    )
```

Sơ đồ trên slide là cây có gốc d; nhánh trái là b với hai con a và c; nhánh phải là e, con phải của e là g, và con trái của g là f.

7.3 Duyệt trung thứ tự

```
inorder Empty = []
inorder (Node value left right) =
  inorder left ++ [value] ++ inorder right
```

```
> inorder tree
"abcdefg"
```

7.4 Chiều cao cây

```
height Empty = 0
height (Node value left right) =
  max (height left) (height right) + 1
```

```
> height tree
4
```

7.5 Hàm bậc cao

Hàm bậc cao là hàm nhận hàm làm tham số hoặc trả về hàm. Với cây nhị phân ở trên, có thể viết một hàm duyệt tổng quát:

```
traverse func zero Empty = zero
traverse func zero (Node value left right) =
  func value
    (traverse func zero left)
    (traverse func zero right)
```

```
height_func _ a b = max a b + 1
```

```
> traverse height_func 0 tree
4
```

Hàm duyệt trung thứ tự cũng có thể được sinh bằng áp dụng một phần:

```
inorder_func value left right = left ++ [value] ++ right
inorder = traverse inorder_func []
```

```
> inorder tree
"abcdefg"
```

8 Ý tưởng hàm trong kỹ nghệ phần mềm

Lập trình hàm không còn bị giới hạn trong các ngôn ngữ ít phổ biến như Haskell. Tư tưởng của nó đã được hầu hết các ngôn ngữ chính thống tiếp nhận. Một số ngôn ngữ hỗ trợ đặc trưng lập trình hàm là Python, JavaScript, Ruby, C++11, C#, Scala.

8.1 Hàm ẩn danh

Ví dụ hàm bình phương trong một số ngôn ngữ:

```
-- Python
lambda x : x**2
```

```
-- C#
x => x**2
```

```
-- JavaScript
function (x) { return x * x; }
```

```
-- C++11
[](int x) -> int { return x * x; }
```

8.2 Bao đóng

Bao đóng cho phép hàm bên trong giữ quyền truy cập đến môi trường nơi nó được tạo ra:

```
function make_closure() {
  var inner_variable = 0;
  return function () {
    return inner_variable++;
  }
}
```

```
var counter = make_closure();
counter(); // 0
counter(); // 1
```

8.3 Dùng bao đóng để currying

Nhiều ngôn ngữ không hỗ trợ áp dụng một phần trực tiếp, vì cơ chế currying xung đột với cách chúng xử lý danh sách tham số. Tuy nhiên có thể dùng bao đóng để mô phỏng. Ví dụ áp dụng một phần cho hàm lũy thừa:

```
function pow5(x) {  
    return Math.pow(x, 5);  
}  
  
pow5(2); // prints 32
```

9 Kết luận

Lập trình hàm bắt đầu từ một lý thuyết rất nhỏ: biểu thức, hàm, gọi hàm và thay thế. Từ lõi đó có thể xây dựng điều kiện, đệ quy, điểm bất động và một mô hình tính toán tương đương máy Turing. Trong thực hành, cùng những ý tưởng ấy xuất hiện dưới dạng giá trị bất biến, hàm bậc cao, khớp mẫu, danh sách vô hạn, bao đóng và hàm ẩn danh.

Tài liệu tham khảo

- [Fixed-point combinator](#)
- [Cantor, Godel, Turing: an eternal golden diagonal](#)
- [Godel incompleteness theorems and agnosticism](#)
- [Ninety-Nine Haskell Problems](#)
- [MDN: Closures](#)
- [C++11 lambda closures](#)